

<百度地图MCP> 路线规划+<百度地图JS API Three> 三维动态可视化

前言

本项目创新性地结合百度地图MCP路线规划能力与百度地图JS API Three的可视化能力，构建了一个路线规划可视化案例，并用广州到从化为例进行展示。案例使用大语言模型结合MCP完成了路线规划，并用JS完成了动态的3D地图线路可视化，支持卫星地图、真是地形、路径飞线、3D人物移动以及相机视角跟随。

案例效果

小红书视频：

- [三维卫星地图动态路线规划，来啦！](#)
- [地图MCP+JS|一站式完成路线规划加可视化](#)
- [当一回荔枝使|地图MCP还原《长安的荔枝》](#)
- [用JS API和MCP在网页地图上画爱心！](#)
- [地图MCP+JS API=可视化旅游攻略！](#)

相关技术

- 百度地图MCP
- 百度地图JS API Three
- JavaScript
- React
- Three.js
- Cesium

技术实现

MCP路径规划

先在Cursor或其他平台使用百度地图MCP进行路线规划。要求：路线详细+真实（路线在道路上+将路线写入 `data/lychee.geojson` 文件中 配置AK：[百度MCP API文档](#)

数据文件 `data/lychee.geojson`

类似这样的结构，coordinates为路线

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "LineString",
        "coordinates": [
```

```
        [113.27345, 23.13538],
        .....
        [113.59358, 23.55379]
    ]
},
"properties": {
    .....
}
}
]
```

AK配置

需要使用百度地图MCP、百度地图JS API Three以及Cesium的AK AK获取：

- [百度地图开放平台](#)
- [Cesium官网](#)

引擎与底图

初始化底图引擎，使用卫星+真实地形底图（BingImageryTileProvider + CesiumTerrainTileProvider）

```
const engine = new mapvthree.Engine(ref.current, {
  map: {
    provider: null,
    center: [113.27345, 23.13538],
    heading: 0,
    pitch: 60,
    range: 2000,
    projection: 'EPSG:4326',
  },
  rendering: {
    enableAnimationLoop: true,
  },
});
let terrain = new mapvthree.CesiumTerrainTileProvider();
let imagery = new mapvthree.BingImageryTileProvider();
let vector = new mapvthree.BaiduVectorTileProvider();
engine.add(new mapvthree.MapView({
  terrainProvider: terrain,
  imageryProvider: imagery,
  vectorProvider: null,
}));
console.log('terrainProvider:', terrain);
console.log('imageryProvider:', imagery);
console.log('vectorProvider:', vector);
```

飞线设置

对地图飞线进行设置

```
const line = engine.add(new mapvthree.FatLine({
  lineWidth: 10,
  keepSize: true,
  color: '#87CEFA',
}));

const flyline = engine.add(new mapvthree.FatLine({
  color: '#ff0000',
  lineWidth: 10,
  keepSize: true,
  lineCap: 'round',
  enableAnimation: true,
  enableAnimationChaos: true,
  animationTailType: 1,
  animationTailRatio: 0.2,
  animationIdle: 1000,
  animationSpeed: 1000,
  emissive: new THREE.Color(0xcf9c00),
}));
```

3D人物模型加载

加载动态3D人物模型，需要提前将模型放入public/models/running_man.glb

```
const loader = new GLTFLoader();
console.log('开始加载模型...');
loader.load(runningManModel, (gltf) => {
  console.log('模型加载成功:', gltf);
  model = gltf.scene;
  model.scale.set(modelScale, modelScale, modelScale);

  // 创建动画混合器
  mixer = new THREE.AnimationMixer(model);
  console.log('动画数量:', gltf.animations.length);

  // 播放所有动画
  if (gltf.animations.length > 0) {
    const runAction = mixer.clipAction(gltf.animations[0]);
    runAction.play();
  }

  engine.add(model);
},
// 加载进度回调
(progress) => {
  console.log('加载进度:', (progress.loaded / progress.total * 100) +
    '%');
},
```

```
// 错误回调
(error) => {
  console.error('模型加载错误:', error);
});
```

动画设置

创建动画，包括人物移动平滑、相机跟随、相机平滑等操作

```
function animate() {
  const currentTime = performance.now();
  const deltaTime = Math.min((currentTime - lastTime) / 1000,
MAX_DELTA_TIME);
  lastTime = currentTime;

  if (coords && coords.length > 0 && model) {
    if (mixer) {
      mixer.update(deltaTime);
    }

    let currentPosOutput = [];
    engine.map.projectArrayCoordinate(coords[currentPointIndex],
currentPosOutput);
    const nextIndex = (currentPointIndex + 1) % coords.length;
    let nextPosOutput = [];
    engine.map.projectArrayCoordinate(coords[nextIndex],
nextPosOutput);

    const distance = Math.sqrt(
      Math.pow(nextPosOutput[0] - currentPosOutput[0], 2) +
      Math.pow(nextPosOutput[1] - currentPosOutput[1], 2)
    );

    const progressDelta = Math.min((MOVEMENT_SPEED * deltaTime) /
distance, 0.1);
    progress += progressDelta;

    const targetX = currentPosOutput[0] + (nextPosOutput[0] -
currentPosOutput[0]) * progress;
    const targetY = currentPosOutput[1] + (nextPosOutput[1] -
currentPosOutput[1]) * progress;

    // 使用真实高程数据进行插值
    const currentHeight = coords[currentPointIndex][2];
    const nextHeight = coords[nextIndex][2];
    const targetZ = currentHeight + (nextHeight - currentHeight) *
progress;

    const direction = new THREE.Vector3(
      nextPosOutput[0] - currentPosOutput[0],
      nextPosOutput[1] - currentPosOutput[1],
```

```

        0
    ).normalize();
    const targetAngle = Math.atan2(direction.y, direction.x) + Math.PI
/ 2;

    if (lastModelPos.x === 0 && lastModelPos.y === 0 && lastModelPos.z
=== 0) {
        lastModelPos.set(targetX, targetY, targetZ);
        lastModelAngle = targetAngle;
    } else {
        lastModelPos.x += (targetX - lastModelPos.x) * 0.1;
        lastModelPos.y += (targetY - lastModelPos.y) * 0.1;
        lastModelPos.z = targetZ;

        let angleDiff = targetAngle - lastModelAngle;
        if (angleDiff > Math.PI) angleDiff -= Math.PI * 2;
        if (angleDiff < -Math.PI) angleDiff += Math.PI * 2;
        lastModelAngle += angleDiff * 0.1;
    }

    if (model) {
        model.position.copy(lastModelPos);
        model.rotation.set(0, 0, 0);
        model.rotateZ(lastModelAngle);
        model.rotateX(Math.PI / 2);
    }

    const targetCameraPos = new THREE.Vector3(
        lastModelPos.x - direction.x * CAMERA_DISTANCE + direction.y *
CAMERA_OFFSET,
        lastModelPos.y - direction.y * CAMERA_DISTANCE - direction.x *
CAMERA_OFFSET,
        lastModelPos.z + CAMERA_HEIGHT
    );

    if (!currentCameraPos) {
        currentCameraPos = targetCameraPos.clone();
    }

    currentCameraPos.lerp(targetCameraPos, CAMERA_SMOOTH);

    engine.camera.position.copy(currentCameraPos);
    engine.camera.lookAt(lastModelPos.x, lastModelPos.y,
lastModelPos.z + 5);
    engine.camera.up.set(0, 0, 1);

    if (progress >= 1) {
        progress = 0;
        currentPointIndex = nextIndex;
    }
}
animationFrameId = requestAnimationFrame(animate);
}

```

数据加载

加载data/lychee.geojson中的数据，绘制飞线、调用动画

```
async function loadData() {
  try {
    // 读取geojson文件
    const response = await fetch('data/lychee.geojson');
    const geojsonData = await response.json();

    // 设定高程
    const terrainProvider = await Cesium.createWorldTerrainAsync();
    // 将坐标转换为Cartographic对象列表
    const positions =
      geojsonData.features[0].geometry.coordinates.map(coord =>
        Cesium.Cartographic.fromDegrees(coord[0], coord[1])
      );
    try {
      const updatedPositions = await
        Cesium.sampleTerrainMostDetailed(terrainProvider, positions, true);
      // 更新geojson中的坐标，添加高程信息
      geojsonData.features[0].geometry.coordinates =
        geojsonData.features[0].geometry.coordinates.map((coord, index) => {
          return [coord[0] - map_bias_x, coord[1] - map_bias_y,
            updatedPositions[index].height + 25];
        });
      console.log(geojsonData.features[0].geometry.coordinates);
    }
    catch (error) {
      console.error('Error sampling terrain:', error);
    }

    // 使用修改后的geojson数据创建数据源
    const dataSource =
      mapvthree.GeoJSONDataSource.fromGeoJSON(geojsonData);
    flyline.dataSource = dataSource;
    line.dataSource = dataSource;

    // 保存坐标数据用于动画
    coords = geojsonData.features[0].geometry.coordinates;

    // 延时2秒后开始动画，等待底图渲染
    console.log('等待2秒让底图渲染...');
    setTimeout(() => {
      console.log('开始动画');
      animate();
    }, 3000);
  } catch (error) {
    console.error('Error loading data:', error);
  }
}
```

```
loadData();
```

释放资源

```
return () => {  
  if (animationFrameId) {  
    cancelAnimationFrame(animationFrameId);  
  }  
  if (mixer) {  
    mixer.stopAllAction();  
  }  
  engine.dispose();  
};
```

参考资料

- [百度地图开放平台](#)
- [JS API Three官方文档](#)
- [百度MCP API文档](#)
- [Cesium开发文档](#)

代码

- [[Cesium开发文档](#)](https://github.com/TianweiGan/BaiduJsThree/tree/master/lychee)